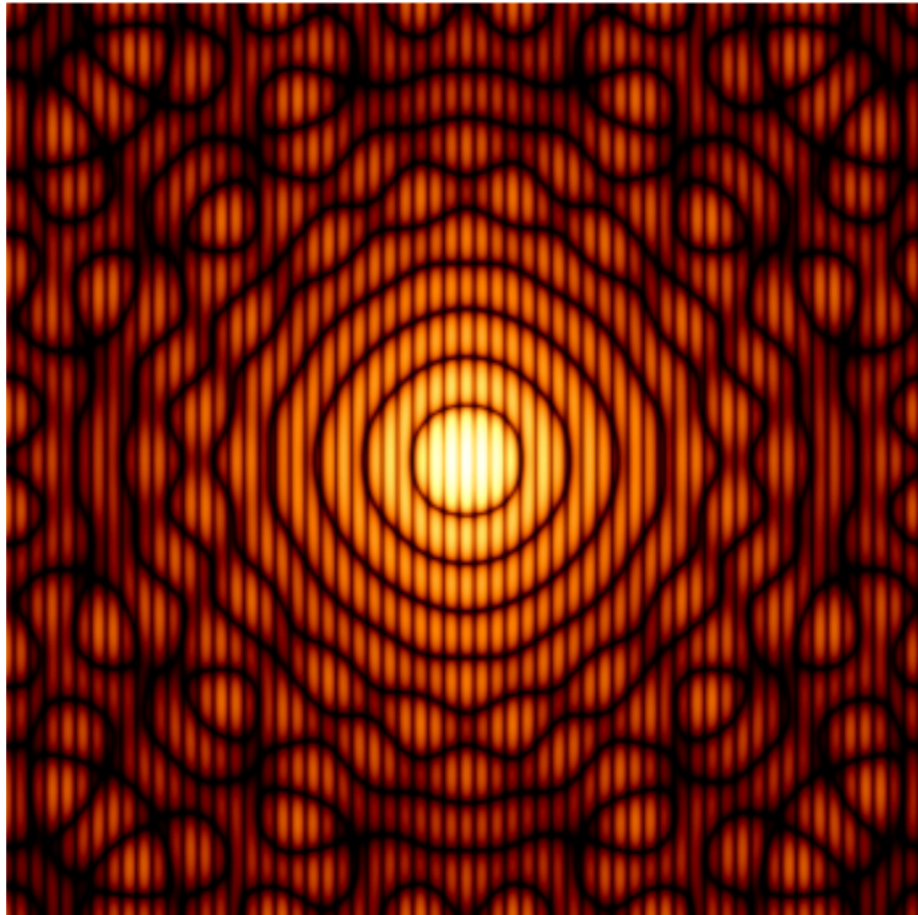


# Projet Méthode Numérique

## “ Figures de diffraction ”

(trou carré, circulaire, double trou)



*Amaury MULLONI*

*Rares BUZAN*



29/01/2026

# Chapitre 1 : Introduction

L'objectif est d'étudier et de simuler numériquement le phénomène de diffraction d'une onde plane à travers différentes ouvertures bidimensionnelles. Dans le régime de la diffraction de Fraunhofer (champ lointain), l'amplitude de l'onde diffractée est proportionnelle à la transformée de Fourier de la fonction de transmission de l'ouverture.

L'enjeu principal est de passer d'un formalisme continu à un formalisme discret exploitable par ordinateur, en mettant en œuvre l'algorithme de la Transformée de Fourier Rapide (FFT) basé sur le lemme de Danielson-Lanczos

Le calcul direct d'une Transformée de Fourier Discrète (DFT) pour une grille de  $N \times N$  points possède une complexité algorithmique de  $O(N^2)$  (soit  $N^2$  pour chaque dimension). Pour une grille de  $512 \times 512$ , cela représenterait des milliards d'opérations, rendant le calcul extrêmement lent. L'utilisation de la FFT permet de réduire cette complexité à  $O(N^2 \log N)$ , rendant la simulation quasiment instantanée.

## Chapitre 2: Fondements théoriques

Cette section présente les bases mathématiques nécessaires au passage du signal continu (l'ouverture physique) au signal discret traité par l'ordinateur, ainsi que l'algorithme d'optimisation utilisé.

### 2.1 Echantillonnage et discrétisation

Pour traiter informatiquement une fonction continue  $f(t)$  (représentant ici la transmission de l'ouverture), nous devons la réduire à un ensemble fini de valeurs. Nous définissons un pas de discrétisation  $s$  tel que  $0 < s \leq 1$ .

L'échantillon discret est alors constitué de la suite  $\{f_k = f(t_k)\}$  ou  $t_k = ks$

La validité de cette discrétisation est régie par le Théorème de Nyquist-Shannon. Ce théorème stipule que si la fonction  $f$  a une bande de fréquence limitée à une fréquence de coupure  $\nu_c = 1/(2s)$ , alors elle est entièrement déterminée par son échantillon discret  $\{f_k\}$ . Dans notre simulation, cela implique que les détails de l'ouverture (comme les bords nets) ne doivent pas varier plus vite que la résolution de notre grille ne le permet.

### 2.2 La Transformée de Fourier discrète (DFT)

L'objectif est de calculer les composantes fréquentielles du signal échantillonné. Sur une grille de  $N$  points (où  $N = 512$  dans notre cas), la Transformée de Fourier Discrète est définie par l'équation (1. 3) du cours:

$$\hat{f}_n = \sum_{k=0}^{N-1} \exp(-i \frac{2\pi}{N} nk) f_k$$

Cette opération permet de passer de l'espace spatial  $\{f_k\}$  à l'espace fréquentiel  $\{\hat{f}_n\}$ . L'implémentation directe de cette somme pour chaque  $n$  et chaque  $k$  nécessite un nombre d'opérations de l'ordre de  $N^2$  (plus précisément  $N(2N - 1)$  opérations complexes). Pour  $N = 512$ , cela reste calculable, mais devient prohibitif pour des dimensions supérieures sans optimisation.

## 2.3 Le lemme de Danielson-Lanczos

Pour optimiser ce calcul, nous utilisons l'algorithme de la Transformée de Fourier Rapide (FFT), qui repose sur une approche récursive. Cette approche est formalisée par le **Lemme de Danielson-Lanczos** (Proposition 1.2 du cours).

Le principe est de diviser la somme de la DFT en deux parties : les indices pairs ( $2j$ ) et les indices impairs ( $2j + 1$ ). Si  $N$  est une puissance de 2 ( $N = 2^L$ ), nous pouvons écrire:

$$\hat{f}_n = F_n^{(0)} + u^n F_n^{(1)}$$

Où:

- $F^{(0)}$  est la DFT de la sous-séquence des points pairs  $f_{2j}$
- $F^{(1)}$  est la DFT de la sous-séquence des points impairs  $f_{2j+1}$
- $u = \exp(-i \frac{2\pi k}{N})$  est le facteur de phase complexe (le signe dans l'exponentielle dépend de la convention directe/inverse choisie)

Cette décomposition permet de ramener le calcul d'une transformée de taille  $N$  à deux transformées de taille  $N/2$ . En appliquant ce lemme récursivement jusqu'à atteindre des échantillons de taille 1 (où la transformée est l'identité), la complexité globale de l'algorithme passe de  $O(N^2)$  à  $O(N \log_2 N)$

## 2.4 Extension à 2 Dimensions

Notre simulation portant sur des ouvertures 2D (images), nous exploitons la propriété de séparabilité de la Transformée de Fourier. La DFT 2D peut être calculée en appliquant successivement :

1. Une FFT 1D sur chaque ligne de l'image.
2. Une FFT 1D sur chaque colonne du résultat précédent. Cette méthode permet de réutiliser directement l'algorithme 1D optimisé décrit ci-dessus

## 2.4 Interprétation physique: Conservation de l'Énergie

Le lien entre l'objet mathématique et la physique est assuré par la relation de Parseval-Plancherel, qui exprime la conservation de l'énergie (ou de la probabilité) entre l'espace temporel/spatial et l'espace fréquentiel. L'intensité lumineuse observée sur un écran de diffraction est proportionnelle au carré du module de l'amplitude de l'onde :

$$I(v) \propto |\hat{f}(v)|^2$$

C'est cette grandeur ( $|\hat{f}|$ ) que nous calculerons et visualiserons pour obtenir les figures de diffraction.

# Chapitre 3 : Implémentation du Code et Résultats

## 3.1 Structure du Code

L'implémentation de la simulation des figures de diffraction repose sur l'utilisation de Python et des bibliothèques NumPy et Matplotlib pour gérer les calculs sur les nombres complexes et la visualisation des résultats. Une implémentation manuelle de l'algorithme de Transformée de Fourier Rapide (FFT) est utilisée pour effectuer le traitement du signal en respectant la théorie du cours. L'affichage a été travaillé pour donner une compréhension très simple des phénomènes d'interférence grâce à une échelle logarithmique. Le programme suit une structure modulaire, avec des fonctions distinctes pour la création de chaque type d'ouverture (carré, cercle, double trou) et pour le calcul mathématique, ce qui permet une organisation claire et une réutilisation facile des composants du code. Vous pouvez très facilement modifier les paramètres du système et vous amuser à observer les résultats.

## 3.2 Définition des Paramètres Globaux

Les principaux paramètres définis dans le programme pour la simulation optique sont :

### Paramètres de la simulation

- **N = 512** : Taille de la grille de simulation (nombre de pixels en largeur et hauteur). Cette valeur doit impérativement être une puissance de 2 ( ici  $2^9$  ) pour permettre le fonctionnement de l'algorithme récursif de FFT (Cooley-Tukey).
- **centre = N // 2 (= 256 ici)** : Coordonnée centrale de l'image, utilisée comme point de référence pour placer les ouvertures.
- **demi, rayon, écart** : Paramètres géométriques définissant respectivement la demi-taille du carré, le rayon du cercle et l'espacement des trous d'Young.

On a choisi une résolution de 512 x 512 car elle offre un compromis optimal entre la précision visuelle des franges de diffraction et la rapidité d'exécution de l'algorithme récursif en Python. Les coordonnées **x** et **y** sont générées via **np.ogrid** pour faciliter la création des masques 2D dans les cas des trous circulaires.

## 3.3 Fonctions Principales

Le programme repose sur la ré-implémentation manuelle de la Transformée de Fourier Rapide (FFT) pour transformer l'ouverture (espace réel) en figure de diffraction (espace des fréquences).

### 3.3.1 Transformée de Fourier 1D (Récursive)

Cette fonction implémente l'algorithme de Cooley-Tukey. Elle divise récursivement le problème en sous-problèmes plus petits (pairs et impairs) jusqu'à atteindre une taille élémentaire, puis recombine les résultats via le "Papillon" (Butterfly).

Définir **fft\_1d(x)** :

- ❖ Récupérer la taille N du vecteur d'entrée.
- ❖ **Cas de base** : Si  $N \leq 1$ , retourner le vecteur tel quel (la transformée d'un point est lui-même).
- ❖ **Division** :
  - Extraire les éléments aux indices pairs pour créer le sous-vecteur **even**.
  - Extraire les éléments aux indices impairs pour créer le sous-vecteur **odd**.
  - Appeler récursivement **fft\_1d** sur ces deux sous-vecteurs.
- ❖ **Calcul du facteur "Twiddle"** :
  - Générer les indices k de 0 à N/2.
  - Calculer le terme de phase complexe  $u = \exp(-2i\pi k / N)$ .
- ❖ **Combinaison** :
  - Calculer le terme **terme\_impair** en multipliant **odd** par le facteur u.
  - Construire la première moitié du résultat : **even + terme\_impair**.
  - Construire la seconde moitié du résultat : **even - terme\_impair**.

- ❖ Retourner la concaténation des deux moitiés.

Note technique : L'implémentation utilise un facteur de phase  $u = \exp(-2i\pi k / N)$  (signe moins) pour respecter la définition de la DFT directe (Eq 1.3 du cours), adaptant ainsi l'algorithme générique présenté en cours.

### 3.3.2 Transformée de Fourier 2D (Séparabilité)

Cette fonction étend la FFT 1D aux images en exploitant la propriété de séparabilité de la transformée de Fourier. Elle évite d'avoir à écrire un algorithme 2D complexe en traitant successivement les lignes puis les colonnes.

Définir **fft\_2d(image)** :

- ❖ **Traitement des lignes :**
  - Pour chaque ligne de l'image, appliquer **fft\_1d**.
  - Stocker le résultat dans une matrice intermédiaire.
- ❖ **Transposition :**
  - Pivoter la matrice (les lignes deviennent des colonnes) pour pouvoir réutiliser la fonction 1D.
- ❖ **Traitement des colonnes :**
  - Pour chaque colonne (maintenant disposée en ligne), appliquer **fft\_1d**.
- ❖ **Transposition inverse :**
  - Pivoter à nouveau la matrice finale pour retrouver l'orientation originale de l'image.
- ❖ Retourner la matrice complexe résultante.

## 3.4 Calcul et Analyse des Résultats

### 3.4.1 Calcul de l'Intensité Diffractée

Cette fonction fait le lien entre les mathématiques (FFT) et la physique (Optique). Elle transforme le champ électrique complexe calculé en une intensité lumineuse observable.

Définir **calcul\_diffraction(image)** :

- ❖ Appeler la fonction **fft\_2d** sur l'image de l'ouverture (le masque).
- ❖ Appliquer un décalage (**fftshift**) pour placer la fréquence nulle (le centre de la diffraction) au milieu de l'image car la DFT standard place la fréquence 0 à l'indice 0 (coin de l'image)
- ❖ Calculer le module au carré de l'amplitude complexe pour obtenir l'intensité physique ( $I = |A|^2$ ).
- ❖ Retourner la matrice d'intensité.

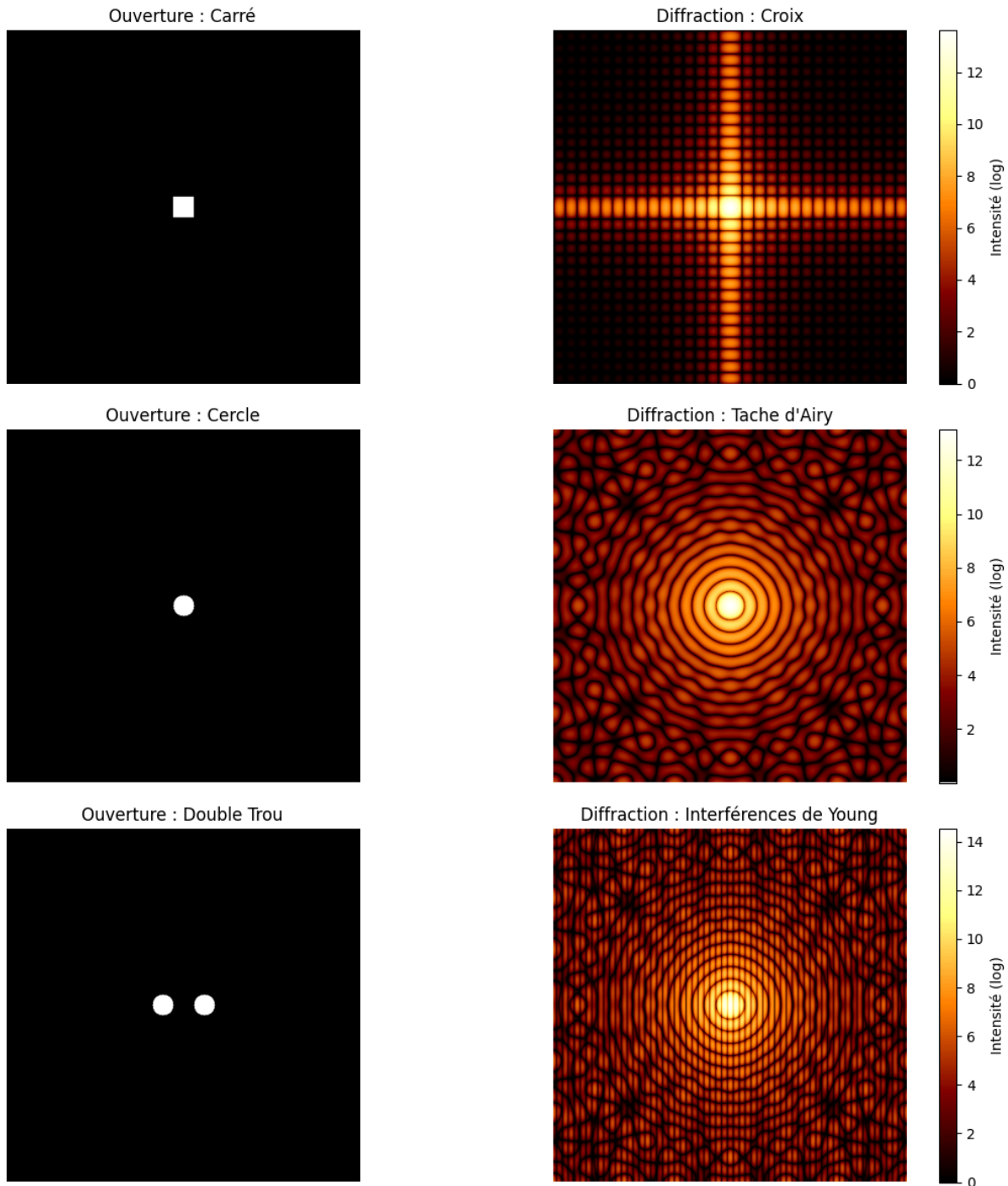
### 3.4.2 Création des Masques et Simulation

Le programme construit trois scénarios classiques d'optique ondulatoire en remplissant des tableaux de zéros (noir) avec des zones de valeur 1 (transparent).

- ❖ **Trou Carré :** On définit une zone centrale carrée par découpage d'indices (slicing).
  - Résultat attendu : Une figure en forme de croix (sinus cardinal 2D).
- ❖ **Trou Circulaire :** On utilise l'équation du cercle  $(x - x_c)^2 + (y - y_c)^2 \leq R^2$  pour créer un masque booléen.
  - Résultat attendu : Une tache centrale entourée d'anneaux concentriques (Tache d'Airy).
- ❖ **Double Trou (Fentes d'Young) :** On définit deux zones circulaires identiques décalées symétriquement par rapport au centre.
  - Résultat attendu : Des anneaux de diffraction modulés par des franges d'interférences rectilignes parallèles.

L'affichage final utilise une échelle logarithmique (**np.log**) pour la visualisation, car l'intensité du pic central est plusieurs ordres de grandeur supérieure à celle des franges secondaires.

### 3.5 Visualisation des Résultats



### 3.6 Analyse et Interprétation des Résultats Visuels

Les figures de diffraction obtenues par notre simulation numérique montrent une excellente concordance avec les prédictions de l'optique ondulatoire théorique. Un principe général s'observe pour toutes les géométries : la largeur de la tache de diffraction est inversement proportionnelle à la dimension de l'ouverture. Ce phénomène illustre le principe d'incertitude : le confinement spatial de la lumière par les bords de l'ouverture entraîne systématiquement un étalement de l'énergie dans l'espace des fréquences.

L'utilisation de l'échelle logarithmique pour l'affichage de l'intensité est ici cruciale pour observer cet étalement : elle permet de révéler les maxima secondaires (les anneaux ou les lobes) qui sont typiquement des milliers de fois moins lumineux que le pic central et seraient invisibles sur une échelle linéaire.

## 1. Le Trou Carré :

La figure de diffraction obtenue présente une symétrie caractéristique en forme de croix. On observe un pic central intense entouré de lobes secondaires s'étendant selon les axes horizontaux et verticaux.

- **Interprétation physique :** Ce résultat correspond parfaitement au produit de deux fonctions sinus cardinal ( $\text{sinc}(x) \cdot \text{sinc}(y)$ ). La symétrie de la croix reflète directement la géométrie anguleuse de l'ouverture.

## 2. Le Trou Circulaire :

La simulation révèle une tache centrale circulaire brillante entourée d'une série d'anneaux concentriques d'intensité décroissante.

- **Interprétation physique :** Il s'agit de la célèbre Tache d'Airy. Mathématiquement, la Transformée de Fourier d'une fonction porte circulaire est une fonction de Bessel du premier type ( $J_1(x)/x$ ). La présence et la régularité des anneaux confirment que notre algorithme FFT gère correctement la symétrie radiale, même sur une grille cartésienne.

## 3. Le Double Trou (Trous d'Young) :

C'est le résultat le plus complexe et le plus riche. On retrouve la structure globale de la diffraction par un trou circulaire (l'enveloppe), mais celle-ci est striée verticalement par des franges noires et brillantes alternées.

- **Interprétation physique :** Cette image illustre la combinaison de deux phénomènes : la diffraction (qui donne la forme globale des cercles, due à la taille finie de chaque trou) et l'interférence (qui crée les stries verticales, dues à la distance séparant les deux trous). C'est la signature visuelle du terme en  $\cos^2$  typique de l'expérience d'Young. Le fait que les franges soient verticales indique que les trous sont alignés horizontalement.

## 3.6 Conclusion

Ce projet avait pour objectif de simuler le phénomène physique de la diffraction de Fraunhofer en exploitant la puissance du calcul numérique. Pour ce faire, nous avons dû franchir le pont entre la théorie de l'optique ondulatoire et l'algorithmique appliquée.

D'un point de vue théorique, nous avons confirmé que la figure de diffraction en champ lointain est l'image mathématique de l'ouverture dans l'espace des fréquences, liée par la Transformée de Fourier. La discrétisation du problème nous a imposé de respecter les conditions de Nyquist-Shannon pour éviter les artefacts de repliement spectral (aliasing).

D'un point de vue technique, l'implémentation de l'algorithme de la Transformée de Fourier Rapide (FFT) a été l'étape clé. En appliquant le lemme de Danielson-Lanczos, nous avons réduit la complexité du calcul de  $O(N^2)$  à  $O(N^2 \cdot \log N)$ . Sur notre grille de 512 x 512 points, cela représente un gain de performance considérable, transformant un calcul qui aurait pris plusieurs minutes (ou heures) par méthode naïve en une exécution quasi instantanée.

Enfin, la visualisation des résultats valide l'approche : nous avons retrouvé avec précision les signatures visuelles du sinus cardinal (carré), de la tache d'Airy (cercle) et des interférences d'Young. Ce travail illustre parfaitement comment les méthodes numériques, lorsqu'elles sont optimisées, permettent de modéliser fidèlement des phénomènes physiques complexes.

Ce fut très enrichissant pour nous de constater que l'on pouvait modéliser de la physique réaliste aussi facilement en mettant en application directe les concepts mathématiques abstraits vus en cours.

## **Bibliographie :**

- **Cours :**

Professeur C.Oguey. Cours “Méthodes numériques pour les matériaux”

- **Algorithmes et Implémentation :**

Transformée de Fourier rapide (FFT) - Wikipédia ( Consulté le 23/01/2026 )

[https://en.wikipedia.org/wiki/Fast\\_Fourier\\_transform](https://en.wikipedia.org/wiki/Fast_Fourier_transform)

Python diffraction and interference's documentation ( Consulté le 23/01/2026 )

<https://diffractio.readthedocs.io/en/latest/#>

Algorithme de Cooley-Tukey - Wikipédia ( Consulté le 24/01/2026 )

[https://en.wikipedia.org/wiki/Cooley-Tukey\\_FFT\\_algorithm](https://en.wikipedia.org/wiki/Cooley-Tukey_FFT_algorithm)

- **Programmation et Bibliothèques :**

Documentation officielle Python 3 :

<https://docs.python.org/3/> ( Consulté le 23/01/2026 )

Documentation NumPy (Gestion des tableaux et nombres complexes)

<https://numpy.org/doc/stable/> ( Consulté le 23/01/2026 )

Documentation Matplotlib (Affichage imshow et échelles de couleurs) :

<https://matplotlib.org/stable/> ( Consulté le 23/01/2026 )

- **Livres (Aide mémoire Python) :**

BOB CORDEAU et LAURENT POINTAL . Python 3 : apprendre à programmer en python . Edition Dunod . 2017

Python Crash Course, 3rd Edition

- **Logiciel utilisé :**

VSCode, Google Colab, Spyder ( Python 3 )